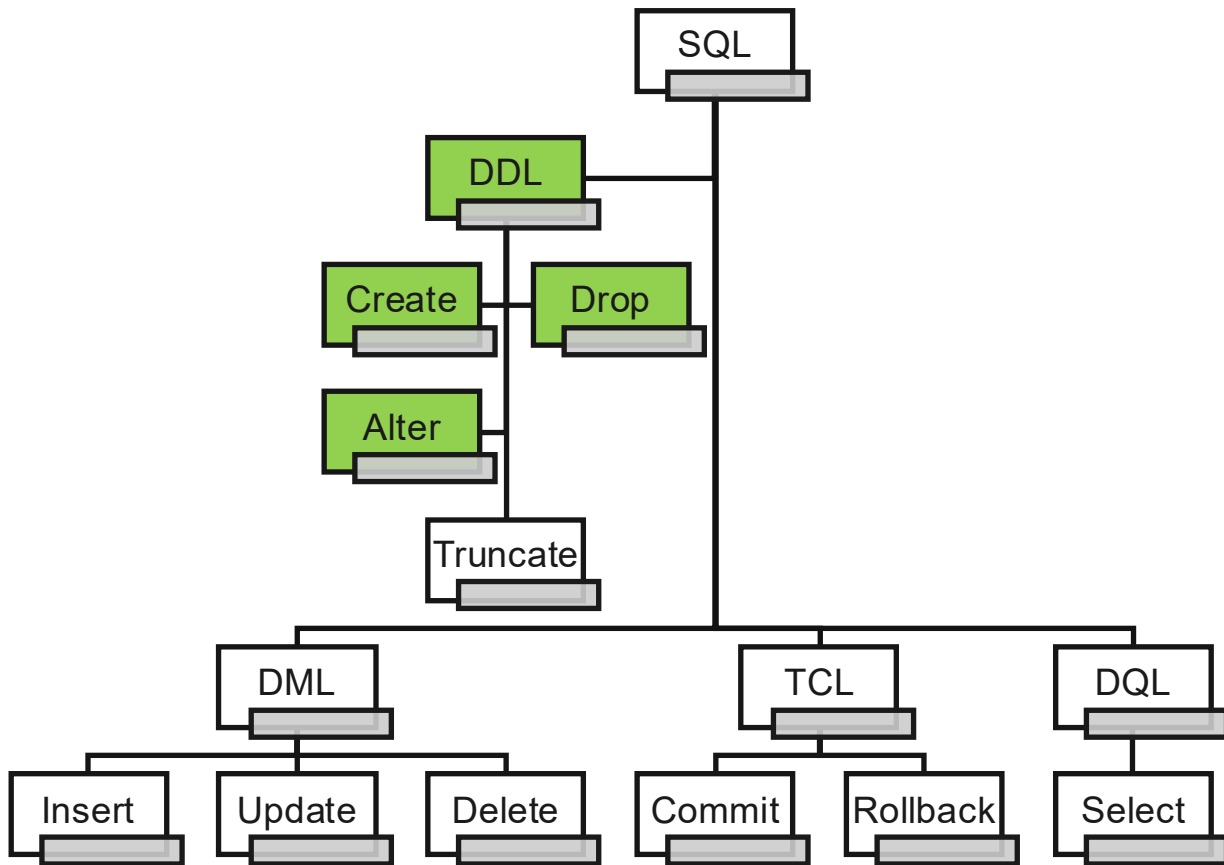


SQL: DDL

Vanessa Braganholo

vanessa@ic.uff.br

Na Aula de Hoje



DDL

ou “criando tabelas”

Vamos usar o SQLite

O SQLite não é um servidor: é uma biblioteca embarcada

- Não há processo separado, nem servidor para instalar, nem usuários para criar
- O programa abre o arquivo .db direto e conversa com ele

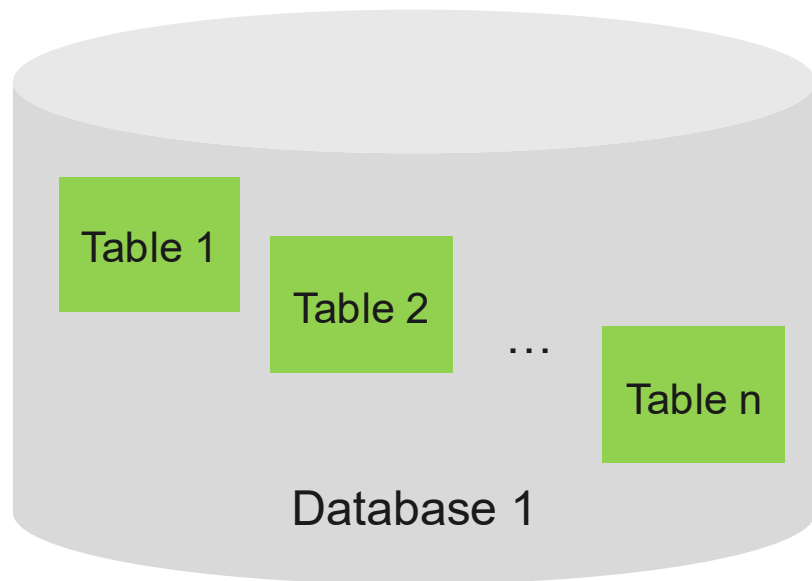
É de domínio público: não tem licença nem dono

É o banco de dados mais implantado do mundo

- Está em praticamente todo celular, navegador, avião e sistema operacional

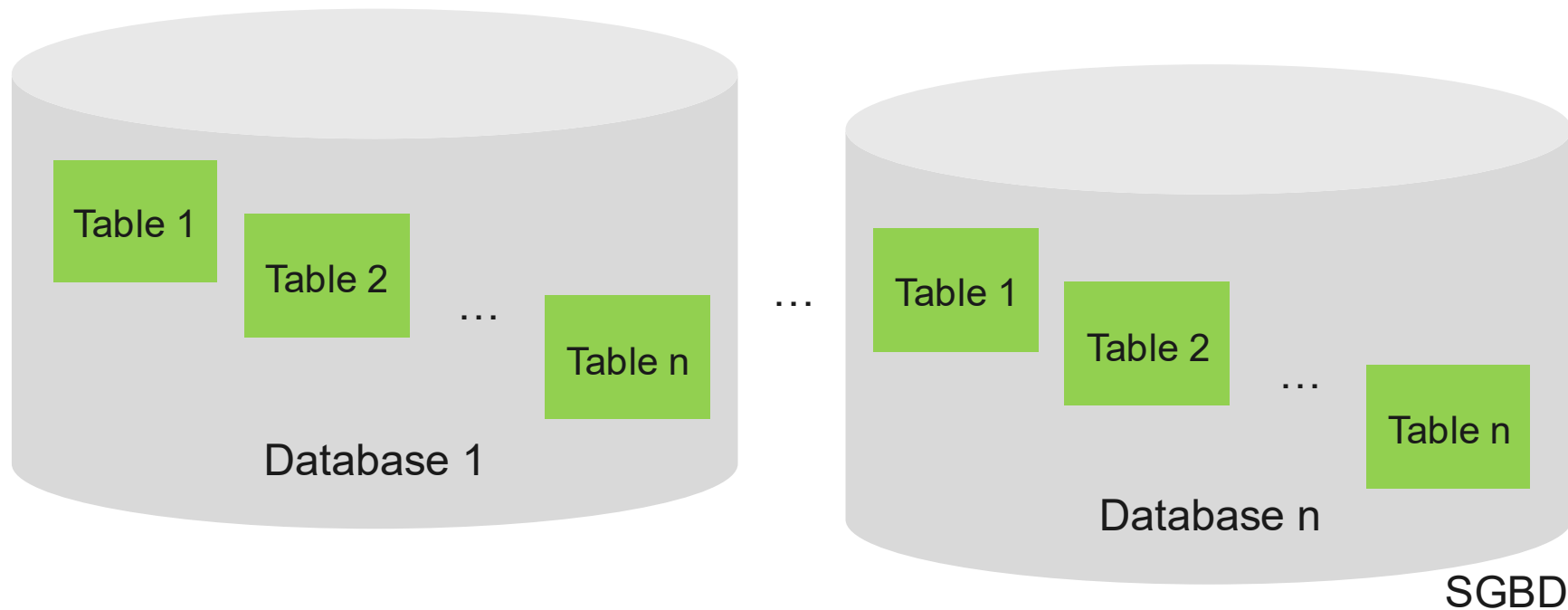


SGBD, Database, Tables...



SGBD

SGBD, Database, Tables...



SGBD

Database

Database

- Identificado por um nome
- Inclui descritores para cada elemento

Elementos incluem

- Tabelas, restrições, views, domínios e outras construções

No SQLite, um database é um arquivo .db

- Não existe identificador de autorização, nem usuários: quem controla o acesso é o sistema de arquivos

CREATE

Instruções CREATE

- Principal comando SQL para a definição de dados

CREATE Database

Instrução CREATE DATABASE (em alguns SGBDs, CREATE SCHEMA)

```
CREATE DATABASE EMPRESA AUTHORIZATION 'vanessa';
```

*usuário “dono”
do esquema*

O SQLite não tem esse comando

- O database é o próprio arquivo, que nasce quando um .db que ainda não existe é aberto

Catálogo

Catálogo

- Coleção nomeada de databases em um ambiente SQL

Ambiente SQL

- Instalação de um SGBDR compatível com SQL

No SQLite o catálogo é uma tabela comum, consultável com SQL

- A consulta abaixo devolve o CREATE TABLE original de cada tabela do banco:

```
SELECT name, sql FROM sqlite_master;
```

SGBD, Database, Tables...



...



SGBD

Como isso fica no SQLite

Um arquivo .db é um database inteiro

- Não existe a camada do servidor: o SQLite é uma biblioteca que abre o arquivo direto
- Todas as tabelas ficam dentro desse arquivo

Como isso fica no SQLite

Para trabalhar com mais de um banco ao mesmo tempo

```
ATTACH DATABASE 'outro.db' AS outro;
```

- **main** é nome do principal

O catálogo é a tabela `sqlite_master`, dentro do próprio arquivo

- Desde a versão 3.33 ela também atende pelo nome `sqlite_schema`

CREATE Table

```
CREATE TABLE <nome da tabela>  
    (<definições de colunas>  
     <definição da chave primária>  
     <definições de chaves alternativas>  
     <definições de chaves estrangeiras>);
```

Criando Tabelas

```
CREATE TABLE aluno
(
    id integer NOT NULL,
    nome varchar(200) NOT NULL,
    endereco varchar(200),
    data_nasc date,
    PRIMARY KEY (id)
);
```

Tipos de Dados

Tipos de dados básicos:

- Tipos de dados numérico:
 - Incluem números: INTEGER ou INT e SMALLINT
 - Números de ponto flutuante (reais): FLOAT ou REAL e DOUBLE PRECISION
- Tipos de dados de cadeia de caracteres:
 - Tamanho fixo: CHAR(*n*) ou CHARACTER(*n*)
 - Tamanho variável: VARCHAR(*n*) ou CHAR VARYING(*n*) ou CHARACTER VARYING(*n*)

Tipos de Dados

- Tipos de dados de cadeia de bits:
 - Tamanho fixo: BIT(n)
 - Tamanho variável: BIT VARYING(n)
- Tipo de dado booleano:
 - Valores TRUE ou FALSE ou NULL
- Tipo de dados DATE:
 - Dez posições
 - Componentes são DAY, MONTH e YEAR na forma DD-MM-YYYY

Tipos de Dados

Tipos de dados adicionais:

- Tipo de dado Timestamp (TIMESTAMP):
 - Inclui os campos DATE e TIME
 - Mais um mínimo de seis posições para frações decimais de segundos
 - Qualificador opcional WITH TIME ZONE
- Tipo de dado INTERVAL:
 - Especifica valor relativo que pode ser usado para incrementar ou decrementar um valor absoluto de uma data, hora ou timestamp

O SQLite aceita todos esses nomes de tipo — mas não faz nada com eles

Tipos de Dados no SQLite

Qualquer que seja o tipo declarado, o valor é guardado em uma de 5 classes

- NULL — o valor nulo
- INTEGER — número inteiro
- REAL — número de ponto flutuante
- TEXT — cadeia de caracteres
- BLOB — sequência de bytes, guardada exatamente como entrou

Não existem os tipos BOOLEAN, DATE, TIME, TIMESTAMP, INTERVAL nem BIT

- Datas são armazenadas como TEXT no formato 'AAAA-MM-DD'; e booleanos como 0 ou 1

Afinidade de Tipo

O tipo declarado não é uma regra: é uma preferência de conversão, chamada afinidade

- O tamanho é decorativo — VARCHAR(3) aceita um texto de 26 caracteres, sem reclamar
- CREATE TABLE x (a BANANA(7)); é aceito sem erro

A afinidade é deduzida do nome que você escreveu

- Contém INT → INTEGER; contém CHAR, CLOB ou TEXT → TEXT
- Contém BLOB ou nada declarado → BLOB; contém REAL, FLOAT ou DOUBLE → REAL
- Qualquer outro caso → NUMERIC

Escrever VARCHAR(200) documenta a intenção, mas não impede nada

Domínios

Domínio:

- Nome usado com a especificação de atributo
- Torna mais fácil mudar o tipo de dado para um domínio que é usado por diversos atributos
- Melhora a legibilidade do esquema
- Exemplo:

```
CREATE DOMAIN TIPO_CPF AS CHAR(11);
```

SQLite não suporta esse comando

Especificando Restrições

Restrições básicas:

- Restrições de chave e integridade referencial
- Restrições sobre domínios de atributo e NULLs
- Restrições sobre tuplas individuais dentro de uma relação

NOT NULL

- NULL não é permitido como valor do atributo

Valor padrão

- DEFAULT <valor>

Cláusula CHECK

```
Dnumero INT NOT NULL CHECK (Dnumero > 0 AND Dnumero < 21);
```

Constraints

Valor do id precisa ser < 500

```
CREATE TABLE aluno
(
    id integer NOT NULL CHECK (id < 500),
    nome varchar(200) NOT NULL,
    endereco varchar(200),
    data_nasc date,
    PRIMARY KEY (id)
);
...
```

Constraints

Valor do id precisa ser < 500

```
CREATE TABLE aluno
(
    id integer NOT NULL,
    nome varchar(200) NOT NULL,
    endereco varchar(200),
    data_nasc date,
    PRIMARY KEY (id),
    CONSTRAINT ck_id CHECK (id < 500)
);
...
```

Chave Primária

Cláusula **PRIMARY KEY**

- Especifica um ou mais atributos que compõem a chave primária de uma relação

```
Dnumero INT NOT NULL PRIMARY KEY;
```

Unique

Cláusula **UNIQUE**

- Especifica chaves alternativas (secundárias)

```
Dnome VARCHAR(15) UNIQUE;
```

Chave Primária Composta

```
CREATE TABLE turma
(
    id integer NOT NULL,
    id_disciplina integer NOT NULL,
    id_monitor integer,
    PRIMARY KEY (id, id_disciplina),
    FOREIGN KEY (id_disciplina) REFERENCES disciplina (id)
        ON DELETE CASCADE,
    FOREIGN KEY (id_monitor) REFERENCES aluno (id)
        ON DELETE SET NULL
);
```

Chave Estrangeira

Especificar comportamento a ser acionado quando o campo correspondente da tabela referenciada é atualizado (ON UPDATE) ou apagado (ON DELETE)

- **NO ACTION:** proíbe alteração/exclusão
 - **RESTRICT:** proíbe alteração/exclusão
 - **SET NULL:** muda valor do campo para NULL
 - **CASCADE:** propaga a atualização/exclusão
 - **SET DEFAULT:** muda o valor do campo para um valor default que tenha sido especificado na criação do campo
- Comportamento default caso não seja especificado nada: NO ACTION
 - A diferença entre NO ACTION e RESTRICT é o momento: RESTRICT falha na hora, NO ACTION verifica no fim do comando

Chave Estrangeira

```
CREATE TABLE turma
(
    id integer NOT NULL,
    id_disciplina integer NOT NULL,
    id_monitor integer,
    PRIMARY KEY (id, id_disciplina),
    FOREIGN KEY (id_disciplina) REFERENCES disciplina (id)
        ON DELETE CASCADE,
    FOREIGN KEY (id_monitor) REFERENCES aluno (id)
        ON DELETE SET NULL
        ON UPDATE CASCADE
);
```

Chave Estrangeira

Opção: uso de CONSTRAINT

```
CREATE TABLE turma
(
    id integer NOT NULL,
    id_disciplina integer NOT NULL,
    id_monitor integer,
    PRIMARY KEY (id, id_disciplina),
    CONSTRAINT fk_disciplina FOREIGN KEY (id_disciplina)
REFERENCES disciplina (id) ON DELETE CASCADE,
    CONSTRAINT fk_monitor FOREIGN KEY (id_monitor) REFERENCES
aluno (id) ON DELETE SET NULL
    ON UPDATE CASCADE
);
```

Atenção: o SQLite não verifica FK por padrão

A declaração de FK é guardada, mas ignorada em tempo de execução

A verificação vale por conexão e não fica gravada no arquivo .db

- O VSCode abre uma conexão nova a cada Run Query
- Selecione sempre o PRAGMA junto com o comando antes de executar

```
PRAGMA foreign_keys = ON;
```

```
INSERT INTO turma VALUES (1, 99, NULL); -- só agora  
seria rejeitado se 99 não existir como id de disciplina
```

Cuidado!

Em muitos SGBDs, a criação de chaves estrangeiras exige atenção à ordem

- Referências circulares
- Ou referência a uma tabela que ainda não foi criada

A solução é criar as tabelas primeiro e acrescentar as restrições depois, com ALTER TABLE

Cuidado!

No SQLite **o problema não existe** — mas essa solução também não

- O CREATE TABLE aceita FK para tabela que ainda não existe: a referência só é resolvida no INSERT/UPDATE
- Não existe ALTER TABLE ... ADD CONSTRAINT
- Basta criar todas as tabelas antes de manipular os dados

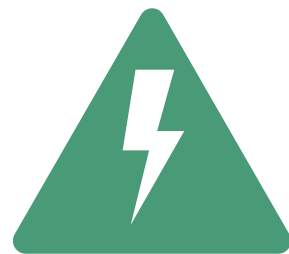
Alteração de Esquema

Comandos de evolução de esquema:

- Pode ser feito enquanto o banco de dados está operando
- Não exige recompilação do esquema

Comandos correspondentes:

- DROP
- ALTER



CUIDADO!!!

Comando DROP

Usado para remover elementos nomeados do database, como tabelas, domínios ou restrições

Exemplo:

```
DROP TABLE aluno;
```

O SQLite não tem DROP DATABASE: apagar o banco é apagar o arquivo .db



CUIDADO!!!

Comando ALTER

Ações de alteração de tabela

- Acrescentar ou remover uma coluna (ADD COLUMN / DROP COLUMN)
- Renomear tabela ou coluna (RENAME TO / RENAME COLUMN)
- Alterar uma definição de coluna (MODIFY)
- Acrescentar ou remover restrições numa tabela (ADD CONSTRAINT, DROP CONSTRAINT)

Comando ALTER no SQLite

MODIFY, ADD CONSTRAINT e DROP CONSTRAINT não existem no SQLite

O DROP COLUMN só existe a partir da versão 3.35 (2021)

Exemplos:

```
ALTER TABLE aluno ADD COLUMN telefone VARCHAR(9);      -- ok
ALTER TABLE aluno DROP COLUMN endereco;                 -- ok
ALTER TABLE aluno MODIFY telefone VARCHAR(11);          -- erro
ALTER TABLE turma DROP CONSTRAINT fk_monitor;           -- erro
```

Exercícios

Vamos usar o SQLite no VSCode ou o SQLiteOnline

O passo a passo da ferramenta está no material SQLite no VSCode

Sem o VSCode? Use o <https://sqliteonline.com/>



Abrir o banco da aula

No Google Classroom, baixe o arquivo aula15.db e salve numa pasta

No VSCode, abra essa pasta pelo menu File / Open Folder

Clique no SQLite Explorer e depois em New Query

Escreva o comando, clique com o botão da direita e escolha Run Query

Apagar tabela DEMO

O arquivo aula15.db vem com uma tabela demo, só para você ter com o que começar

Digite `DROP TABLE demo;` e execute (Run Query)

Confira o resultado com `SELECT name FROM sqlite_master;`

Exercício: Criar tabelas do esquema abaixo

Maquina(CodMag, NomeMag)

-- Tabela com máquinas usadas em uma fábrica para fabricar produtos --

Produto(CodProd, DescrProd, Peso)

-- Tabela com os produtos fabricados --

MaqProd(CodMag, CodProd, Tempo)

CodMag referencia Maquina

CodProd referencia Produto

-- Tabela que informa em que máquinas cada produto pode ser fabricado e qual o respectivo tempo de produção, em minutos --

ComposProd(CodProdComposto, CodProdComponente, Quant)

CodProdComposto referencia Produto

CodProdComponente referencia Produto

-- Tabela que informa a composição de cada produto. Para cada produto, informa quais os produtos que o compõem, e em que quantidade --

Dica: consultar a definição de uma tabela

O SQLite não tem o comando DESCRIBE

Duas formas de ver a definição da tabela ComposProd:

```
PRAGMA table_info(ComposProd); -- uma linha por coluna:  
cid, name, type, notnull, dflt_value, pk
```

```
SELECT sql FROM sqlite_master WHERE name = 'ComposProd';  
-- devolve o CREATE TABLE original, inteiro
```

Exercício

1. Adicionar restrição na tabela Produto, para que Peso tenha que ser > 0
2. Adicionar restrição na tabela MaqProd, para que tempo tenha que ser > 0
3. Excluir a restrição adicionada no exercício 1.
4. Alterar a tabela Produto para adicionar uma coluna PRECO do tipo FLOAT.
5. Excluir a coluna quant da tabela ComposProd.
6. Excluir a tabela ComposProd.
7. Mudar o tipo da coluna nomeMaq da tabela Maquina para VARCHAR(50).

Por que 1, 2, 3 e 7 não funcionam

O SQLite só tem quatro formas de ALTER TABLE

- ADD COLUMN, DROP COLUMN, RENAME COLUMN e RENAME TO
- O DROP COLUMN é o mais novo dos quatro: só a partir da versão 3.35

Não é uma falha do SQL

- Em MySQL, PostgreSQL ou Oracle, os quatro exercícios funcionam como estão escritos
- É uma escolha de projeto do SQLite: um banco embarcado troca poder de alteração por simplicidade

Para todo o resto, o caminho oficial é recriar a tabela

Recriando a tabela

Roteiro oficial para qualquer alteração que o ALTER TABLE não faz

- Exemplo: acrescentar CHECK (peso > 0) na tabela Produto

```
PRAGMA foreign_keys = OFF;
BEGIN;
CREATE TABLE Produto_novo (
    codProd INTEGER NOT NULL PRIMARY KEY,
    descrProd VARCHAR(30),
    peso FLOAT CHECK (peso > 0));
INSERT INTO Produto_novo SELECT codProd, descrProd, peso FROM Produto;
DROP TABLE Produto;
ALTER TABLE Produto_novo RENAME TO Produto;
COMMIT;
PRAGMA foreign_keys = ON;
```

Exercício: agora recriando a tabela

Refaça, com o roteiro do slide anterior, os exercícios que deram erro:

1. Produto: fazer com que Peso tenha que ser > 0
2. MaqProd: fazer com que Tempo tenha que ser > 0
3. Produto: remover a restrição criada no exercício 1
7. Maquina: mudar o tipo de nomeMaq para VARCHAR(50)

Em todos, os dados que já estavam na tabela devem continuar lá

Resposta: exercício 1

2, 3 e 7 seguem o mesmo roteiro — no 2, lembre de repetir as duas chaves estrangeiras

```
PRAGMA foreign_keys = OFF;
BEGIN;
CREATE TABLE Produto_novo (
    codProd INTEGER NOT NULL PRIMARY KEY,
    descrProd VARCHAR(30),
    peso FLOAT CHECK (peso > 0));
INSERT INTO Produto_novo SELECT codProd, descrProd, peso FROM Produto;
DROP TABLE Produto;
ALTER TABLE Produto_novo RENAME TO Produto;
COMMIT;
PRAGMA foreign_keys = ON;
```

Referência

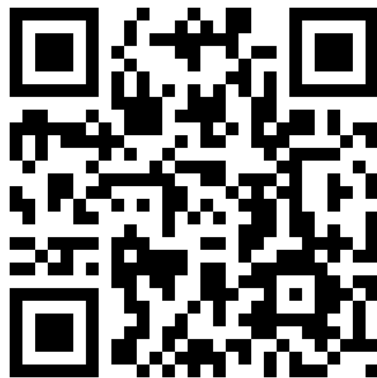
Sistemas de Banco de Dados. Ramez E. Elmasri, e Shamkant Navathe, 2005, 4a edição, Ed. Pearson - Addison Wesley.

Referência

Tutorial SQLite:

<https://www.sqlitetutorial.net/>

Sintaxe: <https://www.sqlite.org/lang.html>



Referência

W3 Schools (apresenta sintaxe para vários BDs diferentes)

- <https://www.w3schools.com/sql/default.asp>



Agradecimento

Slides baseados no material de Daniel Oliveira

Perguntas?

Vanessa Braganholo

vanessa@ic.uff.br